

Resumen General del Laboratorio

Mostrar de forma práctica y real un pipeline de **DevSecOps** donde la seguridad se integra desde el primer `commit`.

Flujo que se demuestra:

1. Commit → GitHub Actions se dispara automáticamente.
2. **SAST** con **CodeQL** (análisis estático de código).
3. Build de imagen Docker.
4. Escaneo de seguridad de contenedor con **Trivy**.
5. Si hay vulnerabilidades **HIGH/CRITICAL** → el pipeline **falla**.
6. Al corregir las vulnerabilidades → pipeline pasa y se publica la imagen en **GHCR**.

Tecnologías utilizadas:

- Node.js + Express (aplicación sencilla con XSS)
- Docker
- GitHub Actions
- CodeQL (SAST)
- Trivy (escaneo de imágenes)
- GitHub Container Registry (ghcr.io)

Conceptos clave a transmitir:

- Shift-left security
- Automatización de seguridad en el pipeline
- Fail-fast (fallar rápido ante problemas graves)
- Diferencia entre detectar y bloquear

Preparación inicial

1. Clonar el repositorio (cambia por tu usuario)

```
git clone https://github.com/rafasgomez/devsecops-demo.git
cd devsecops-demo
```

2. Crear las carpetas necesarias

```
mkdir -p .github/workflows
npm install
```

PARA el fork con SBOM

```
git clone -b feature/sbom-axios-vulnerable https://github.com/rafasgomez/devsecops-demo.git
cd dir
npm install
node app.js
```

Estructura final del proyecto

text

```
devsecops-demo/  
├── app.js  
├── package.json  
├── Dockerfile  
├── .github/  
│   └── workflows/  
│       └── devsecops-pipeline.yml
```

1. package.json

```
{  
  "name": "devsecops-demo",  
  "version": "1.0.0",  
  "main": "app.js",  
  "scripts": {  
    "start": "node app.js"  
  },  
  "dependencies": {  
    "express": "^4.19.2"  
  }  
}
```

2. app.js – Dos versiones

Versión Vulnerable:

```
const express = require('express');  
const app = express();  
const port = 3000;  
  
app.get('/', (req, res) => {  
  const name = req.query.name || 'World';  
  // ❌ Vulnerable a XSS Reflejado  
  res.send(`<h1>Hello, ${name}</h1>`);  
});  
  
app.listen(port, () => {  
  console.log(`🚀 App listening on http://localhost:${port}`);  
});
```

Versión Segura (corrige la vulnerabilidad):

```
const express = require('express');  
const app = express();  
const port = 3000;  
  
function escapeHtml(unsafe) {
```

```

return unsafe
  .replace(/&/g, "&amp;")
  .replace(/</g, "&lt;")
  .replace(/>/g, "&gt;")
  .replace(/"/g, "&quot;")
  .replace(/'/g, "&#039;");
}

app.get('/', (req, res) => {
  const name = req.query.name || 'World';
  // ✅ Corregido: sanitizamos la entrada
  res.send('<h1>Hello, ${escapeHtml(name)}!</h1>');
});

app.listen(port, () => {
  console.log(`🚀 App listening on http://localhost:${port}`);
});

```

3. Dockerfile

```

FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force
COPY . .
EXPOSE 3000
USER node
CMD ["npm", "start"]

```

```

# Imagen base oficial de Node.js (versión 20) basada en Alpine Linux
# Alpine es ligera → reduce superficie de ataque y tamaño de imagen
FROM node:20-alpine

```

```

# Establece el directorio de trabajo dentro del contenedor
# Todas las instrucciones siguientes se ejecutarán desde /app
WORKDIR /app

```

```

# Copia únicamente los ficheros de dependencias primero
# (package.json y package-lock.json si existe)
# Esto permite aprovechar la cache de Docker en builds posteriores
COPY package*.json ./

```

```

# Instala solo dependencias de producción (no devDependencies)
# npm ci → instalación reproducible basada en package-lock.json
# Limpia la cache para reducir tamaño de imagen

```

```
RUN npm ci --only=production && npm cache clean --force
```

```
# Copia el resto del código de la aplicación al contenedor
```

```
# Se hace después para no invalidar la cache de dependencias
```

```
COPY . .
```

```
# Expone el puerto 3000 (no abre realmente el puerto)
```

```
EXPOSE 3000
```

```
# Cambia al usuario no privilegiado "node"
```

```
# Evita ejecutar el contenedor como root → mejora seguridad
```

```
USER node
```

```
# Comando por defecto al arrancar el contenedor
```

```
# Ejecuta la aplicación usando npm start
```

```
CMD ["npm", "start"]
```

4. Pipeline final

Archivo: `.github/workflows/devsecops-pipeline.yml`

name: DevSecOps Pipeline - SAST + Docker Security (con fail en graves)

on:

push:

branches: [`main`]

jobs:

pipeline:

runs-on: `ubuntu-latest`

permissions:

contents: `read`

security-events: `write`

packages: `write`

steps:

- **name:** Checkout code

uses: `actions/checkout@v4`

- **name:** Setup Node.js

uses: `actions/setup-node@v4`

with:

node-version: `'20'`

- **name:** Install dependencies

run: npm ci

===== SAST - CodeQL =====

- name: Initialize CodeQL
uses: github/codeql-action/init@v4
with:
languages: javascript
- name: Perform CodeQL Analysis (SAST)
uses: github/codeql-action/analyze@v4

===== Docker Multi-arch Build =====

- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v3
- name: Login to GitHub Container Registry
uses: docker/login-action@v3
with:
registry: ghcr.io
username: \${{ github.actor }}
password: \${{ secrets.GITHUB_TOKEN }}
- name: Build Docker image (multi-arch)
uses: docker/build-push-action@v6
with:
context: .
push: false
tags: ghcr.io/\${{ github.repository_owner }}/devsecops-demo:latest
platforms: linux/amd64,linux/arm64
cache-from: type=gha
cache-to: type=gha,mode=max
load: true

===== Escaneo con Trivy (falla el pipeline) =====

- name: Scan Docker image with Trivy (falla en HIGH/CRITICAL)
uses: aquasecurity/trivy-action@master
with:
image-ref: 'ghcr.io/\${{ github.repository_owner }}/devsecops-demo:latest'
format: 'table'
exit-code: '1' # ← FALLA si hay HIGH o CRITICAL
severity: 'CRITICAL,HIGH'
ignore-unfixed: true
vuln-type: 'os,library'

===== Push solo si todo OK =====

- name: Push multi-arch image to GHCR
uses: docker/build-push-action@v6
with:
context: .
push: true
tags: ghcr.io/\${{ github.repository_owner }}/devsecops-demo:latest
platforms: linux/amd64,linux/arm64

cache-from: type=gha
cache-to: type=gha,mode=max

Instrucciones rápidas de despliegue del laboratorio

1. Crear repositorio en GitHub (devsecops-demo)
2. Clonar y crear los archivos anteriores
3. Primer commit con versión **vulnerable**
4. git push → observar pipeline y alertas en **Security** → **Code scanning**
5. Corregir app.js con versión segura
6. git push → pipeline debe pasar y publicarse la imagen

Para probar localmente:

```
docker run -p 3000:3000 --rm ghcr.io/rafasgomez/devsecops-demo:latest
```

<https://github.com/rafasgomez/devsecops-demo/pkgs/container/devsecops-demo>

```
# Listar todas las imágenes en GHCR
```

```
docker search ghcr.io/rafasgomez
```

```
# Ver detalles de la imagen
```

```
docker pull ghcr.io/rafasgomez/devsecops-demo:latest
```

```
docker inspect ghcr.io/rafasgomez/devsecops-demo:latest
```

Cómo revisar las alertas de CodeQL en GitHub

1. Ve a tu repositorio: <https://github.com/rafasgomez/devsecops-demo>
2. En el menú superior haz clic en **Security** (Seguridad)
3. En la barra lateral izquierda haz clic en **Code scanning alerts**
4. Allí verás:
 - Alertas en rojo si hay vulnerabilidades (ej: "Reflected cross-site scripting")
 - Severidad (High, Medium, etc.)
 - Detalle de la línea de código afectada

Nota: Las alertas aparecen unos 30-90 segundos después de que termine el workflow.

OTROS COMANDOS Y RECURSOS:

<https://github.com/rafasgomez/devsecops-demo/security/code-scanning>

```
npm install
```

```
git add package-lock.json
git commit -m "Add lockfile"
git push
```

```
% docker pull ghcr.io/rafasgomez/devsecops-demo:latest
http://localhost:3000/?name=rafa%3Cscript%3Ealert
```

```
docker run -p 3000:3000 --rm ghcr.io/rafasgomez/devsecops-demo:latest
docker run -p 3000:3000 ghcr.io/rafasgomez/devsecops-demo:latest
```